

03 - Data Structures

哈尔滨工业大学 黄子扬

2026 年 6 月 6 日

Contents

1	Introduction / 什么是数据结构	2
2	Tuples / 元组 (Parentheses: ())	2
2.1	Properties / 核心属性	2
3	Lists / 列表 (Square Brackets: [])	2
3.1	Mutability / 可变性	2
3.2	List Methods / 列表常用方法	3
3.3	The Reference Pitfall / 引用与拷贝陷阱 (重要考点)	3
4	Dictionaries / 字典 (Curly Braces: {k: v})	3
4.1	Key-Value Pairs / 键值对逻辑	3
5	Advanced: List Comprehensions / 列表推导式	4
6	Summary / 如何选择数据结构? [7, 16]	4

1 Introduction / 什么是数据结构

【Concept / 核心概念】 1

Data Structure A data structure models a collection of data, such as a list of numbers or a record in a database. [1]

数据结构是对数据集合的建模。Python 内置了三种核心结构：元组 (Tuples)、列表 (Lists) 和字典 (Dictionaries)。

2 Tuples / 元组 (Parentheses: ())

2.1 Properties / 核心属性

- **Ordered / 有序**: Elements appear in a fixed sequence. [2]
- **Heterogeneous / 异构**: Can contain different types (e.g., (1, "John")). [2]
- **Index starts at 0**: 索引从 0 开始。[2]

【Warning / 避坑指南】 1

Immutability / 不可变性 Like strings, tuples are **immutable**. You cannot change the value of an element once created. [3]

元组一旦创建就无法修改。试图执行 `t[4] = "David"` 会抛出错误。[5]

【Example / 实操案例】 1

Tuple Basics / 元组基础案例

```
tuple2 = (1, "John", "Red")
print(len(tuple2)) # Output: 3
print(tuple2[4])   # Output: "John"
# Membership check
if "Red" in tuple2:
    print("There is red")
```

3 Lists / 列表 (Square Brackets: [])

3.1 Mutability / 可变性

【Concept / 核心概念】 1

Mutable vs. Immutable Unlike tuples, lists are **mutable**. You can update or alter the data during the program. [6, 7]

列表是 **** 可变 **** 的，你可以随时修改、添加或删除其中的元素。

3.2 List Methods / 列表常用方法

- `.append(x)`: Append to the end. [8]
- `.insert(i, x)`: Insert value `x` at index `i`. [9]
- `.pop(i)`: Remove and return the value at index `i`. [9]
- `.split()`: Create a list from a string (e.g., `"a,b".split(",")`). [6]

【Example / 实操案例】1

List Operations / 列表操作案例

```
mylist = ["A", "C"]
mylist.insert(1, "B") # ["A", "B", "C"]
out = mylist.pop(2)   # out is "C", mylist is ["A", "B"]
mylist.append("D")    # ["A", "B", "D"]
```

3.3 The Reference Pitfall / 引用与拷贝陷阱 (重要考点)

【Warning / 避坑指南】1

Memory Reference / 内存地址引用 Assigning one list to another (`list2 = list1`) does **NOT** create a copy. Both refer to the same object in memory! [10]

直接赋值只是起了个“别名”，修改其中一个，另一个也会跟着变。

【Example / 实操案例】1

Independent Copy / 如何正确拷贝

```
animals = ["lion", "tiger"]
# WRONG way:
large_cats = animals
large_cats.append("Leopard") # BOTH lists now have "Leopard"! [11]

# RIGHT way (using slicing):
large_cats = animals[:]
large_cats.append("Leopard") # Only large_cats is changed. [11]
```

4 Dictionaries / 字典 (Curly Braces: {k: v})

4.1 Key-Value Pairs / 键值对逻辑

【Concept / 核心概念】1

Mapping vs. Sequence Sequence types (Lists/Tuples) use integers as indices. Dictionaries use **unique keys** as labels. [12, 13]

字典通过唯一的“键”来存储和查找“值”，而不是靠位置索引。

【Example / 实操案例】1

Dictionary Management / 字典管理案例

```
capitals = {"France": "Paris", "Spain": "Madrid"}
# Access value
print(capitals["France"]) # Output: "Paris"
# Add new item
capitals["Germany"] = "Berlin"
# Delete item
del capitals["Spain"]
```

【Warning / 避坑指南】1

Existence Check / 存在性检查 Accessing a non-existent key raises a **KeyError**. Always use **in** to check first:

```
if "Spain" in capitals: print(capitals["Spain"]) [14, 15]
```

5 Advanced: List Comprehensions / 列表推导式

【Concept / 核心概念】1

Short-hand for Loops A list comprehension is a succinct way to create a list from an existing iterable. [16, 17]

列表推导式是 **for** 循环的优雅简写形式。

【Example / 实操案例】1

Comprehension vs. Traditional Loop

```
numbers = (1, 2, 3, 4, 5)
# Traditional way:
squares = []
for num in numbers:
    squares.append(num**2)

# Comprehension way (Single line!):
squares = [num**2 for num in numbers] [18]
```

6 Summary / 如何选择数据结构？[7, 16]

- **Use a List**: 数据有自然顺序，且需要经常更新/修改。
- **Use a Tuple**: 数据有自然顺序，但 **** 严禁 **** 修改（保护数据安全）。
- **Use a Dictionary**: 数据无序，核心需求是通过标签（键）快速查找值。